

Project 2: Schema Linking with Supervised Fine-Tuning

CSE/DSC 234 – Data-Centric AI and AI Engineering, Spring 2026

Jay Chaudhary | Abhay Jain

June 4, 2026

1. Training Data Methodology

Base split. We use the released 301 training examples and 101 validation examples from the SNAILS-derived dataset without any external augmentation. The 17 database schemas span national-parks biodiversity, NTSB crash statistics, NY education data, and SAP business-resource planning. We deliberately chose *not* to augment with Spider or synthesised NL-SQL pairs because schema serialisation and prompt design proved to be higher-leverage axes: fixing a broken schema format moved the score by up to 0.4 points, whereas preliminary augmentation runs introduced noise from domain mismatch.

Schema serialisation. Each schema is rendered as one line per table in the format:

```
TABLE(col*, col>RefTable, col, ...)
```

where `*` marks a primary key and `>RefTable` marks a foreign key. No spaces after commas. This encoding fits all relational structure (joins, PK identity) in minimal tokens, keeping 94% of training examples below 2048 tokens without truncation. Serialisation is implemented in `format_data.py::serialize_compact`, reading `primary_keys` and `foreign_keys` directly from Spider-format schema JSON.

Output format. The gold target is the schema-links dict serialised as compact JSON (`separators=(",",":")`). Tables referenced without specific columns (e.g. `COUNT(*)`) are emitted as `{"T": []}`, matching the wildcard-boundary rule in the eval script exactly.

Chat record construction. Each example becomes a three-turn supervised chat record: *system* (35-token compact instruction), *user* (question first, then `DB:` and `Schema:` block), *assistant* (compact JSON). Question-first ordering was motivated by the hypothesis that the decoder should attend to question tokens while scanning each schema line; ablation confirmed a +0.09 score gain. Records are saved as HuggingFace Dataset Arrow files via `format_data.py`.

2. Final Pipeline Architecture

Table ?? summarises the best configuration (C8, score **0.6117**).

Fine-tuning (train.py). LoRA adapters are applied via RapidFire AI’s `RFLoraConfig` and trained with `RFSFTConfig` wrapping HuggingFace TRL’s `SFTTrainer`. Experiments are managed through `Experiment` + `RFGGridSearch`; each config runs to completion (`num_chunks=1`) before checkpoints are compared by validation loss. The best checkpoint is copied to `./adapter/`.

Inference (main.py). The base model is loaded from HuggingFace Hub; the LoRA adapter is attached via `PeftModel.from_pretrained(base, "./adapter")`. Raw model output is parsed by a tolerant extractor: direct `json.loads`, then regex on the first `{...}` block, then empty-dict fallback. `filter_to_schema` removes any hallucinated table or column not present in the actual schema, using case-insensitive lookup maps to restore exact casing. Both adapter location and base model ID are auto-detected from `adapter/adapter_config.json` so the script is self-contained.

Table 1: Final pipeline configuration (C8).

Component	Setting
Base model	Qwen/Qwen2.5-1.5B-Instruct ($\leq 2B$ constraint)
LoRA targets	q_proj, k_proj, v_proj, o_proj
LoRA rank / alpha	$r = 64$, $\alpha = 128$, dropout = 0.1
Learning rate	5×10^{-5} , cosine schedule, 5% linear warmup
Batch / accumulation	1 per device $\times$ 4 gradient-accumulation steps (eff. batch 4)
Epochs / seq. length	5 epochs, max 2048 tokens
Precision	bf16 mixed precision
Schema format	TABLE(col*,col>Ref) compact; PK=*, FK=>Ref
Prompt layout	Question first: Q:{q} then DB:{id}, Schema:{schema}
System prompt	35-token compact instruction (no verbose preamble)
Decoding	Greedy (do_sample=False), repetition penalty 1.1
Post-processing	filter_to_schema: case-insensitive hallucination removal

3. Experimentation Methodology

We ran all configs through RapidFire AI’s `Experiment` / `RGridSearch` API, using `RFLoraConfig`, `RFModelConfig`, and `RFSFTConfig` to define each knob bundle. Configs were compared by validation loss logged to `logs/` and by end-to-end leaderboard score on the 101-question validation set. Underperforming configs were stopped via the IC Ops interface; promising ones were cloned and modified.

Table ?? lists all 9 distinct configurations explored across five knob axes: schema format, learning rate, LoRA rank, prompt layout, and system-prompt wording.

Table 2: All configurations explored. “Meas.” = full train + eval on validation set.

Config	r	LR	Ep.	Schema / Prompt	Status
C1 Baseline	16	2×10^{-4}	3	Simple TABLE(c1,c2), schema-first	Meas.
C2 high LR	64	1×10^{-4}	5	Simple, schema-first	Meas.
C3 low LR	64	8×10^{-6}	5	Simple, schema-first	Meas.
C4 JSON hier.	64	5×10^{-5}	5	Nested JSON with types, PK, FK	Meas.
C5 rich inline	64	5×10^{-5}	5	TABLE(col [PK,TEXT],...)	Meas.
C6 r64 simple	64	5×10^{-5}	5	Simple, schema-first	Meas.
C7 compact+prune	64	5×10^{-5}	5	Compact PK/FK, Q-first, keyword pruning	Meas.
C8 compact (best)	64	5×10^{-5}	5	Compact PK/FK, Q-first, no prune	Meas.
C9 r128+col prompt	128	5×10^{-5}	5	Compact PK/FK, Q-first, col-aware sys. prompt	Meas.

Learning rate search (C1–C3, C6). With 301 training examples and effective batch 4 (75 steps/epoch), LR is the most sensitive knob. At $\geq 10^{-4}$ (C1, C2) token accuracy oscillated between 0.42 and 0.56: the model peaked near step 50 (epoch 1) then degraded — a classic small-dataset overshoot. At 8×10^{-6} (C3) convergence was too slow (score 0.47). 5×10^{-5} with cosine schedule and 5% warmup gave stable, monotonically improving validation loss across all 5 epochs.

Schema format search (C4–C8). Nested JSON (C4) frequently exceeded 2048 tokens on databases with 20+ tables, causing truncation (score 0.20). Rich inline annotation (C5) added too many tokens and confused the model (0.45). The simple TABLE(col1,col2) format (C6) reached 0.54. The compact PK/FK

variant (C7, C8) encodes relational structure in near-zero extra tokens: `col*` for PKs and `col>Ref` for FKs tells the model which columns are join keys without verbose type strings.

Pruning ablation (C7 vs. C8). C7 pruned the schema at inference time to keyword-matching tables plus FK neighbours. Despite reducing median context length, table score dropped from 0.683 to 0.552 because pruning has no oracle knowledge at inference – tables whose names share no lexical tokens with the question (e.g., a lookup table referenced only through a FK chain) were silently hidden. Removing pruning (C8) recovered the table score and set the best overall result.

Rank and column-aware prompt (C9). Doubling the rank to $r = 128$ and adding an explicit column-type enumeration to the system prompt (*“List ALL columns: SELECT outputs, WHERE/HAVING filters, JOIN keys, GROUP BY and ORDER BY columns”*) targeted the column recall gap. Score was 0.6074 – marginally below C8 – suggesting the column gap is limited by data scale rather than adapter capacity or instruction wording at this stage.

4. Results and Analysis

Table 3: Results on the 101-question validation set.

Config	r	LR	Ep.	Table Score	Column Score	Score
C1 Baseline	16	2×10^{-4}	3	–	–	0.520
C2 high LR	64	1×10^{-4}	5	–	–	0.520
C3 low LR	64	8×10^{-6}	5	–	–	0.470
C4 JSON hier.	64	5×10^{-5}	5	–	–	0.200
C5 rich inline	64	5×10^{-5}	5	–	–	0.450
C6 r64 simple	64	5×10^{-5}	5	–	–	0.540
C7 +prune	64	5×10^{-5}	5	0.552	0.484	0.518
C8 (best)	64	5×10^{-5}	5	0.683	0.541	0.612
C9 r128+col	128	5×10^{-5}	5	0.685	0.529	0.607

What worked. The compact PK/FK schema format combined with question-first prompt layout produced the largest single gain (+0.07 over simple format). Embedding relational structure inline improved both table and column scores: the model can infer join paths from `>Ref` markers without a separate “Relationships” section. Stable learning rate (5×10^{-5}) was essential on this data scale – higher rates caused oscillation and lower rates failed to converge in 5 epochs.

What didn’t work. Richer schema formats (C4, C5) hurt rather than helped: JSON hierarchical truncated long schemas, and rich inline annotations added noise tokens without proportional signal gain. Inference-time schema pruning (C7) degraded table recall by hiding tables that had no lexical overlap with the question. Larger adapter rank (C9) and an explicit column-type instruction in the system prompt did not improve column recall, suggesting the gap is data-scale limited.

Table vs. column gap. The table score (0.683) exceeds the column score (0.541) by 0.14 throughout all runs. Column prediction is harder: the model must identify implicit references – join keys, WHERE predicates, GROUP BY columns – that are not lexically present in the question. Zero schema-invalid identifiers were predicted in C8 and C9, confirming that `filter_to_schema` eliminates all hallucinations.

Surprises. Removing schema pruning (C8 vs. C7) *increased* score despite longer contexts. The 2048-token limit was not the bottleneck; hiding relevant tables was. The column-aware system prompt (C9) was expected to lift column recall but had no effect, implying that the model already “knows” to look for filter

and join columns – the problem is that the small training set does not cover enough distinct join patterns to generalise.

With more time/budget. (1) Augment with LLM-generated NL-SQL pairs labelled by `sql_to_schema_links.py`, targeting under-represented SAP sub-modules (~ 6 training examples each). (2) Curriculum training: start with single-table questions, progress to multi-table joins. (3) Sorted output: always emit tables and columns in schema-defined order to reduce decoding variance and improve F_1 . (4) Custom diagnostic metric: *join-coverage score* – fraction of FK-join columns correctly predicted – to target the implicit-reference gap directly.

5. AI Tools Statement

Claude Code (Anthropic) was used throughout this project as a coding assistant: generating boilerplate for `format_data.py`, `train.py`, and `main.py`; debugging schema-parsing index errors; iterating on schema serialisation formats; and drafting this report. All experimental design decisions, hyperparameter choices, result interpretation, and final report content are the responsibility of the author. Training scripts, prompt designs, and pipeline decisions were reviewed, tested, and validated end-to-end by the author on DSMLP before inclusion.

References

- [1] E. J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR 2022*.
- [2] T. Yu et al. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. *EMNLP 2018*.
- [3] K. Luoma and A. Kumar. SNAILS: Schema Naming Assessments for Improved LLM-Based SQL Inference. *ACM SIGMOD 2025*.